Review-First Invoice Intelligence: Combining Deterministic Extraction, Arithmetic Consistency Checks, and Human-in-the-Loop Review for Indian GST Workflows

Akshay Kumar Bolusani
ProjectKaro
Hyderabad, India
akshaykumarbolusani@gmail.com

***Abstract*—Extracting structured data from Indian Goods and Services Tax (GST) invoices is error-prone, and extraction errors carry tax consequences: a wrong tax head, rate, or total propagates into filings. Human review of extracted invoices is therefore mandatory in practice, but reviewers are given raw fields with no signal about which documents deserve scrutiny. We present a review-first invoice intelligence system built on three design rules: deterministic extraction before learned extraction, no inferred values (a field is populated only when the source states it), and Decimal arithmetic consistency checks as a cheap, interpretable review signal. The system pairs a deterministic rules extractor with an optional local large language model (LLM) extractor, validates every extraction with explicit reconciliation checks (line items against subtotal, taxes against grand total, per-line quantity–price products, duplicated-allocation detection, GSTIN format), and routes flagged documents to a human reviewer with the failing checks visible. We evaluate in four explicitly separated evidence classes. On hand-built fixtures (3 documents, 92 scored fields), the rules extractor reproduced all 92 fields, while local LLMs scored 84/92 (Qwen2.5-VL 7B) and 76/92 (Qwen2.5 3B); the dominant LLM failure class was invention of line-level tax allocations the source did not state. On a generated synthetic benchmark (60 documents, 660 scored scalar fields), the rules pipeline reproduced all fields and line items. In a controlled error-injection study over the same 60 ground-truth records, the consistency checks fired their intended signal on every injected arithmetic corruption (6 classes, 60 documents each) and on all duplicated allocations (48 documents), produced no false flags on 150 benign or arithmetically consistent control mutations, and respected their documented Rs. 1.00 tolerance exactly. The checks have a principled blind spot—arithmetically consistent invented allocations pass—which is precisely why the design keeps a human in the loop. All results are synthetic-fixture evidence; accuracy on real invoices is not measured here and is not claimed.**

***Index Terms*—document understanding, invoice extraction, GST, arithmetic consistency, human-in-the-loop, deterministic systems**

I. Introduction

An Indian GST invoice is a dense regulatory artifact: seller and buyer GSTINs, HSN/SAC codes, multiple tax slabs, and three tax heads (CGST, SGST, IGST) whose applicability depends on whether the supply is intra- or inter-state. An extraction error is not a cosmetic defect; it changes the tax a business claims or owes. Consequently, automated invoice processing in GST workflows is, in practice, automation *plus* human review—and the review step is where time goes and where attention is spread uniformly over documents that mostly do not need it.

Learned document-understanding models have made extraction itself increasingly capable [1]–[4]. But a fluent extractor introduces a failure mode that matters specifically for tax documents: it fills fields the source does not state. In our fixture comparisons (Section IV-B), a local vision-language model repeatedly populated per-line tax rates and CGST/SGST amounts that the invoice never printed, by splitting invoice-level totals across lines. The invented values are arithmetically plausible and individually reasonable—and unsupported by evidence. For a tax workflow, an invented-but-consistent value is worse than a missing one, because nothing about it invites a second look.

This paper describes a system designed around that observation, and evaluates the parts of it that can be evaluated honestly today:

- **Deterministic-first extraction.** A rules extractor with an explicit never-invent rule (a generic tax amount is never mapped to a specific tax head without evidence) is the baseline; a local LLM extractor is optional and its output is held to the same standard.
- **Arithmetic consistency as a review signal.** A set of explicit Decimal reconciliation checks converts an extraction into a routing decision: *valid*, *review required*, or *invalid*, with the failing checks named for the reviewer.
- **Evidence-classified evaluation.** We separate synthetic-fixture results, generated-benchmark results, controlled corruption studies, and audits of public parser outputs, and we do not let one stand in for another. Real-world accuracy requires authorized real invoices and is reported as future work with a defined protocol (Section IV-F).

Our contributions are the system design, a fixture-level comparison exposing the invention failure class in local LLM extraction, and a controlled error-injection study measuring

whether the consistency checks fire their intended signals, respect their tolerance, and stay silent on benign variation.

II. Related Work

LayoutLM [1] jointly pre-trains text and layout for document image understanding and established the modern learned-extraction paradigm; LayoutLMv3 [2] unifies text and image masking in a single multimodal transformer. Donut [3] removes the OCR stage entirely with an OCR-free transformer, and DocLLM [4] brings layout awareness to generative language models for enterprise documents, invoices among them. These works optimize extraction quality. Our work is complementary: whatever extractor is used, its output still needs a verification layer before a human signs off on tax-relevant fields, and deterministic reconciliation is a verification layer that learned models do not provide for themselves.

Human-in-the-loop and selective-review designs route uncertain cases to people; our routing signal is not model confidence but failed arithmetic, which has the advantage of being explainable in one sentence to the reviewer who must act on it. Commercial GST-invoice extraction tools exist, and at least one recent academic system addresses GST invoice processing with learned extraction. To scope the novelty claim, we searched IEEE Xplore, ACM Digital Library, SpringerLink, and Google Scholar (October 2026) for combinations of (GST OR "goods and services tax") AND (invoice) AND (arithmetic OR consistency OR validation OR reconciliation) AND (extraction OR review). The search surfaced extraction-focused work but no study that isolates deterministic arithmetic-consistency checks as an evaluated review-routing signal for GST invoices. We therefore claim, hedged accordingly, that the consistency layer as an explicit, separately evaluated signal appears unstudied in the literature we could find—not that invoice validation itself is new.

III. System Design

A. Ingestion and Extraction

Documents (PDF, text, common image formats) are ingested with Docling and RapidOCR. Two extractors produce the same schema: (i) a deterministic rules extractor encoding GST invoice conventions (labeled fields, tax-head keywords, line-item tables), and (ii) an optional OpenAI-compatible local LLM extractor (evaluated with Qwen2.5-VL 7B and Qwen2.5 3B via Ollama). The rules extractor is the default and the reference: it populates a field only when the source states it.

B. Arithmetic Consistency Checks

All money arithmetic uses `Decimal`. The validator checks, at a documented tolerance of Rs. 1.00:

- required fields present (document type, vendor, items, grand total);
- GSTIN format for vendor and buyer (format only; checksum verification is future work, Section V);
- sum of line taxable values equals the subtotal;
- subtotal plus CGST, SGST, and IGST equals the grand total;
- per line, quantity $\times$ unit price equals taxable value;
- line-level tax amounts sum to the invoice-level tax total, and no line carries a copy of the full invoice-level tax (duplicated allocation, a recurring LLM error);
- line-level GST rates, when populated, are noted as requiring source support, since invoice-level totals alone do not establish per-line rates.

Failures are errors (*invalid*); softer anomalies are warnings (*review required*). A tax-review stage clears strong duplicated line-level tax patterns while preserving invoice totals and recording a warning. Nothing is silently corrected: source values, warnings, and reviewer corrections remain visible in the review interface.

C. Review-First Workflow

The reviewer sees the source document, extracted fields, and the named failing checks. A comparison view separates extractor disagreements into missing, substantive, and formatting-only differences, so reviewer effort concentrates on substantive ones.

IV. Evaluation

A. Datasets and Protocols

Table I defines the four datasets once; every result below cites its dataset, its denominator, and whether the metric is field-level (scored fields correct over fields scored) or document-level (documents exactly right over documents). Proportions carry Wilson 95% confidence intervals. All four datasets are synthetic or third-party artifacts; none contains real invoices.

D2's scored scalars are: document type, invoice number, invoice date, vendor name, vendor GSTIN, buyer name, subtotal, total CGST, total SGST, total IGST, grand total. (Buyer GSTIN and currency appear in ground truth but are not among the generator's scored scalars.)

B. Fixture Comparison: Rules vs. Local LLMs

On D1, the rules extractor reproduced every field. Both LLMs lost fields, and the losses were concentrated in one class: *invention of unstated line-level tax allocations*—populating per-line GST rates and CGST/SGST/IGST amounts by splitting invoice-level totals (e.g., assigning IGST of Rs. 180 to a line whose invoice stated tax only in aggregate). The 3B model additionally altered invoice-level tax totals (Rs. 198 stated, Rs. 207 produced). Table II reports field-level accuracy and document-level exact match. The fixture set is small by design—it is a regression set—but the failure class it exposes motivates the architecture: the system that refuses to infer beat the systems that infer fluently, on exactly the fields where inference is dangerous.

LLM mean latencies were 17.5 s (7B) and 8.8 s (3B) per document against a rules warm median of $\approx$0.49 s on the fixtures—a secondary but operationally relevant gap for a review pipeline.

TABLE I
EVALUATION DATASETS. D1–D3 ARE CONSTRUCTED WITH KNOWN LABELS; D4 AUDITS STORED OUTPUTS OF OTHER TOOLS AND IS NOT GROUND TRUTH.

ID	Construction	Size / denominator	Used for
D1	3 hand-built GST fixtures (INV-001/002/003)	34+24+34 = 92 scored fields	Rules vs. LLM
D2	Generated (script + seed 20261015 in repo); 3 layouts (compact/spaced/wrapped, 20 each), 2 tax modes (CGST+SGST, IGST); 1–5 line items	60 documents; 11 named scalar fields × 60 = 660; 180 line items	Rules pipeline
D3	D2 ground-truth records + one deterministic corruption per run	60 records per class (48 for duplicated allocation)	Consistency-c
D4	Stored parser outputs in a public invoice dataset	1,348 records (1,343 parsed)	Motivation; a

TABLE II
D1 FIXTURE COMPARISON (3 DOCUMENTS, 92 SCORED FIELDS). ACCURACY IS FIELD-LEVEL WITH WILSON 95% CIS; EXACT-MATCH IS DOCUMENT-LEVEL. LLM LATENCY IS THE MEAN PER DOCUMENT; RULES LATENCY IS THE REPO-RECORDED WARM MEDIAN ON THE FIXTURES.

Extractor	Fields	Accuracy [95% CI]	Exact docs
Rules (deterministic)	92/92	1.000 [0.960, 1.000]	3/3
Qwen2.5-VL 7B (local)	84/92	0.913 [0.838, 0.955]	1/3
Qwen2.5 3B (local)	76/92	0.826 [0.736, 0.890]	0/3

C. Synthetic Benchmark (D2)

The rules pipeline reproduced all 660 normalized scalar fields (accuracy 1.000, CI [0.994, 1.000]) and all 180 line items (precision/recall/F1 = 1.0, CI [0.979, 1.000]), with 60/60 documents arithmetic-consistent. We independently re-ran the generator with the same seed in a clean environment and reproduced these figures exactly; commands are given in the reproducibility statement. D2 is constructed-label regression evidence: the generator wrote the ground truth, so the result shows the pipeline implements its specification—not that it generalizes to real invoices.

D. Error-Injection Study (D3)

To measure the consistency layer directly, we applied deterministic corruptions to the D2 ground-truth records (one class at a time) and recorded whether the validator fired the *intended* named signal, not merely any flag. Corruption magnitudes (Rs. 50–100) were chosen well above the Rs. 1.00 tolerance to test detection; two boundary probes at Rs. 0.50 and Rs. 2.00 test the tolerance itself. Table III reports results. All arithmetic corruption classes were caught on every applicable document (per-class catch rate 1.000, CI [0.940, 1.000]); the duplicated-allocation warning fired on all 48 applicable documents (CI [0.926, 1.000]). The tolerance behaved as documented: the sub-tolerance probe was flagged on 0/60 documents and the just-above-tolerance probe triggered the intended error on 60/60. Three control classes produced zero false flags in 150 mutations (false-flag rate 0.000, CI [0.000, 0.025]): thousands separators, currency symbols, and—critically—arithmetically consistent invented splits of invoice tax across lines (30 documents), which pass by construction. The checks therefore verify internal consistency, not truth; Section V discusses the consequence.

TABLE III
D3 ERROR-INJECTION STUDY ON THE 60 D2 GROUND-TRUTH RECORDS. "CAUGHT" MEANS THE INTENDED NAMED CHECK FIRED. CATCH RATES CARRY WILSON 95% CIS.

Class	Intended signal	n	Caught [95% CI]
Grand-total drift (+Rs. 50)	error	60	60 [0.940, 1.000]
Subtotal drift (+Rs. 100)	error	60	60 [0.940, 1.000]
Line taxable drift (+Rs. 100)	error	60	60 [0.940, 1.000]
Tax-total drift (+Rs. 100)	error	60	60 [0.940, 1.000]
GSTIN format break	error	60	60 [0.940, 1.000]
Missing grand total	error	60	60 [0.940, 1.000]
Duplicated allocation	warning	48	48 [0.926, 1.000]
Tolerance probes (grand-total drift)			
+Rs. 0.50 (below tolerance)	none expected	60	0 flagged
+Rs. 2.00 (above tolerance)	error	60	60 caught
Controls (expected: no flag)			
Invented consistent split	—	30	0 flagged
Thousands separators	—	60	0 flagged
Currency symbols	—	60	0 flagged

E. Audit of Public Parser Outputs (D4)

As motivating evidence, we audited 1,348 stored records of a public invoice dataset: 1,343 parsed as JSON, and 1,326 of the parsed records (98.7%) failed at least one arithmetic check. Whatever produced those outputs, they are pervasively inconsistent in exactly the ways Section III-B checks for—evidence that inconsistency in stored extraction outputs is common in the wild, and that such outputs should not be treated as ground truth. D4 says nothing about our system's accuracy.

F. Real-Invoice Study: Protocol and Status

The study that would support deployment claims is defined but not yet run; we report its protocol here and its results nowhere, because no real-invoice data has been collected for this paper. Protocol: 50–100 GST invoices used with the documented permission of their owners; sampling across vendors, layouts, and both tax modes; ground truth produced by manual transcription against the document with written annotation rules (a field is labeled only as stated; unstated tax splits are labeled absent, never inferred); ambiguous fields adjudicated by a second annotator; metrics are field-level extraction accuracy per extractor, consistency-flag false-positive rate on clean extractions, and reviewer time per invoice with versus without flags under counterbalanced ordering. Confidential invoices are never uploaded to public repositories or third-party services;

only aggregate metrics would be published. Until that study runs, every accuracy statement in this paper is scoped to synthetic evidence.

V. Limitations

All accuracy figures in this paper are synthetic-fixture evidence; real-world accuracy is unmeasured and unclaimed. The injection study uses designed single-field corruptions of clean synthetic documents, so its perfect catch rates (with the stated confidence intervals) validate the implementation against its specification—they are not an estimate of error detection on real invoices, whose errors are messier and compounded. The consistency checks cannot detect arithmetically consistent invention, as the control class demonstrates; this is a design boundary, not a bug, and it is why review remains human. GSTIN validation is format-only; the GSTIN check digit is not verified. The fixture comparison involves three documents and two local models; latency figures are single-machine measurements. The system does not certify GST compliance, and reconciled arithmetic does not prove a tax rate or allocation is correct.

VI. Conclusion

For GST invoice workflows, the valuable automation is not extraction alone but extraction whose output arrives with its own audit: deterministic where possible, silent where the source is silent, and flagged by named arithmetic failures where it is inconsistent. Our fixture and injection results show such a layer can be built, behaves to specification within measured confidence intervals, and exposes a concrete failure class of local LLM extractors—invented tax allocations—that fluent extraction benchmarks do not penalize. The open work is empirical: executing the real-invoice protocol of Section IV-F and timing reviewers with and without the flags.

Data and Code Availability

The system source code, the D2 generator with its fixed seed, the fixture set D1 with recorded extractor outputs, the D3 injection script with its raw results, and the D4 audit outputs are available in the project repository (github.com/AkshayKumarBolusani/India-Invoice) and in the frozen release archive accompanying this paper; code is released under the MIT license. No real invoice data is included in any release.

Acknowledgment

The author thanks the reviewers of earlier drafts for pressing on dataset separation and denominators. In accordance with IEEE policy on AI-generated content (PSPB Operations Manual 8.2.1.B.10), the author discloses that Muse (Meta) was used for drafting assistance and for implementing the evaluation tooling described in Section IV; all system design, experiments, data, and results are the author's own, and the author takes full responsibility for the content.

References

[1] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of Text and Layout for Document Image Understanding," in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD)*, 2020, doi: 10.1145/3394486.3403172.

[2] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking," in *Proc. 30th ACM Int. Conf. on Multimedia*, 2022, arXiv:2204.08387.

[3] G. Kim *et al.*, "OCR-free Document Understanding Transformer," in *Proc. European Conf. Computer Vision (ECCV)*, 2022, arXiv:2111.15664.

[4] D. Wang *et al.*, "DocLLM: A Layout-Aware Generative Language Model for Multimodal Document Understanding," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, vol. 1, 2024, pp. 8529–8548, arXiv:2401.00908.